

Guide pratique — corrigé exécutable de l'atelier

Pas-à-pas animateur : toutes les commandes, tous les résultats attendus

A. EL HAJJAJI

Ce guide est le **compagnon d'exécution** du [cahier_atelier.md](#) : pour chaque séquence, les commandes exactes à taper, le résultat attendu (vérifié sur cette plateforme), et le corrigé de chaque exercice. À dérouler tel quel pour préparer l'atelier ou pour le rejouer en autonomie.

Conventions :

- Toutes les commandes se lancent depuis la racine du projet (`/di/dev/data/dbt_benefit`), sauf mention contraire.
 -  **Attendu :** décrit ce que vous devez observer. Si vous observez autre chose → annexe B du cahier (dépannage).
 -  **Reset :** après chaque démo destructive, `make seed && make build` remet la plateforme dans l'état nominal.
-

0. Préparation (à faire AVANT le jour 1)

```
cd /di/dev/data/dbt_benefit
```

```
# 0.1 – Vérifier que la base source tourne (elle est fournie par benefits-dataset)
docker ps | grep pocs
```

✅ Attendu : `pocs-postgres` (port 5433) et `pocs-pgadmin` (port 5050) sont Up.

```
# 0.2 – Environnement
cp .env.example .env          # les valeurs par défaut pointent déjà sur pocs-
postgres:5433
```

```
# 0.3 – Plateforme complète (≈ 2-3 min ; Airflow peut attendre le jour 3)
docker compose --profile "*" up -d
docker compose --profile "*" ps
```

✅ Attendu : tous les services Up ; `benefits-airflow` peut mettre 2-3 min de plus (il installe dbt au premier boot).

```
# 0.4 – Outillage Python
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pre-commit install
```

```
# 0.5 – Sanity check dbt
cd dbt && dbt deps && dbt debug && cd ..
```

✅ Attendu : `dbt deps` installe `dbt_utils`, `dbt_expectations`, `elementary` ; `dbt debug` finit par « **All checks passed!** ».

```
# 0.6 – Premier chargement + premier build (à faire une fois, pour amorcer
#       les tables Elementary et le journal d'audit)
make seed && make build
```

✅ Attendu : `Done. PASS=71 WARN=18 ERROR=0 SKIP=0 TOTAL=89` — **18 warnings, 0 erreur**. Les 18 warnings sont les anomalies volontaires des seeds : c'est l'état nominal de l'atelier.

JOUR 1

S1 — Métier : corrigé de l'exercice pgAdmin (§1.6)

Ouvrir <http://localhost:5050> (identifiants pgAdmin du repo benefits-dataset), serveur → base `pocs` → schéma `src`. Requêtes corrigées :

```
-- Q1 : dossier le plus nombreux
select numero_dossier, count(distinct numero_individu) as nb_membres
from src.dossier group by 1 order by 2 desc limit 1;
-- → DOS-2020-001 (famille Dupont)

-- Q2 : prestations orphelines (annonce RG_23)
select distinct p.numero_individu
from src.prestation p
left join src.individu i using (numero_individu)
where i.numero_individu is null;
-- → au moins IND-999 selon le contenu de src (dans les seeds dbt : IND-999)

-- Q3 : permis en chevauchement (annonce RG_29)
select a.numero_individu, a.permis, b.permis
from src.permis a join src.permis b
on a.numero_individu = b.numero_individu
and a.date_debut < b.date_debut
and coalesce(a.date_fin, '9999-12-31') >= b.date_debut;
```

i Selon le contenu réel de `src.*`, Q2/Q3 peuvent différer des seeds dbt — c'est un point pédagogique : l'atelier travaille ensuite sur les **seeds**, copie contrôlée avec cas piégés garantis.

S2 — Le contrat : rien à exécuter, deux fichiers à ouvrir

```
code dbt/dbt_project.yml dbt/contracts/prestation.yml
```

Corrigé de l'exercice « qualifier des règles » (§2.5) : 1 = déclaratif/alerte (plafond au contrat, RG_15)
· 2 = algorithmique/bloquant (RG_17) · 3 = conformité/obligatoire (RG_34) · 4 = déclaratif/bloquant
(regex au contrat, RG_28) · 5 = algorithmique/bloquant (RG_24).

S3 — Plateforme : tour du propriétaire

```
docker exec -it pocs-postgres psql -U user -d pocs -c '\dt src.*'      # 4 tables
docker exec -it pocs-postgres psql -U user -d pocs -c '\dn'           # schémas :
après le 0.6,                                                         #
bronze/silver/gold/audit/snapshots existent
```

Puis ouvrir chaque URL du tableau §3.3 du cahier et cocher.

S4 — Bronze : traçabilité et qualité technique

```
# 4.1 – Recharger la zone Bronze simulée et regarder RG_01
make seed
docker exec -it pocs-postgres psql -U user -d pocs \
  -c "select numero_individu, _ingested_at, _source_system, _batch_id
      from bronze.individu where numero_individu='IND-003';"
```

✓ Attendu : **2 lignes** pour IND-003 — une `postgres_src` (06:00) et une `redpanda_cdc` (07:30, email différent). C'est le doublon volontaire qui justifiera RG_03.

```
# 4.2 – RG_02 : fraîcheur
cd dbt && dbt source freshness && cd ..
```

✓ Attendu : statut **WARN** sur les 4 sources dès que les seeds ont plus de 24 h (leur `_ingested_at` est figé). Démo : `make freshen` puis relancer → **PASS**. C'est exactement ce que verrait l'exploitation si NiFi s'arrêtait.

```
# 4.3 – RG_30/31/32 : le système d'alarme de l'ingestion
cd dbt
dbt test --select tag:rg30      # volumétrie minimale → PASS (les seeds dépassent
les seuils)
dbt test --select tag:rg31      # anomalies de volume Elementary → WARN possible
(peu d'historique : normal, en discuter)
dbt test --select tag:rg32      # dérive de schéma → PASS (le schéma n'a pas bougé)
cd ..
```

Démo RG_30 qui bloque (option animateur) :

```
docker exec -it pocs-postgres psql -U user -d pocs -c "truncate bronze.prestation;"
cd dbt && dbt test --select tag:rg30 && cd ..      # → FAIL sur prestation : 0 < 10
attendues
make seed                                          # 🔄 reset
```

Démo CDC (§4.4) — nécessite `wal_level=logical` sur la source :

```
docker exec -it pocs-postgres psql -U user -d pocs -c "show wal_level;"
# si 'logical' :
curl -X POST http://localhost:8083/connectors -H "Content-Type: application/json" \
  -d @ingestion/debezium/connector-postgres-benefits.json
# puis provoquer un UPDATE (cahier §4.4) et observer http://localhost:8086
# si 'replica' : faire la démo sur le service postgres du compose (déjà configuré
wal_level=logical)
```

JOUR 2

S5 — Silver

5.1 – Construire la zone Silver

```
cd dbt && dbt build --select staging && cd ..
```

- ✓ Attendu : ~10 warnings (RG_03 n'en produit pas — elle corrige ; RG_06, 09, 10, 13, 14, 28, 29, 31 alertent).

5.2 – Vérifier RG_03 : le doublon IND-003 a disparu, la version CDC a gagné

```
docker exec -it pocs-postgres psql -U user -d pocs \  
-c "select count(*) from silver.stg_individu;" \  
-c "select numero_individu from silver.stg_individu where numero_individu='IND-003';"
```

- ✓ Attendu : 11 individus (12 lignes bronze – 1 doublon), IND-003 une seule fois.

Corrigé de l'exercice pivot (§5.3) — LE moment clé de l'atelier :

1. Dans `dbt/dbt_project.yml`, bloc `vars.contracts.prestation.plafonds`, ajouter :

```
FORFAIT_INTEGRATION: 1000
```

- 2.
- ```
cd dbt
dbt compile --select stg_prestation -q
grep -A8 "is_montant_suspect"
target/compiled/benefits_lakehouse/models/staging/stg_prestation.sql
```

- ✓ Attendu : le CASE compilé contient une branche `when type_prestation = 'FORFAIT_INTEGRATION' and montant_prestation > 1000` — **générée**, pas écrite.

- 3.
- ```
dbt build --select stg_prestation+ && cd ..
```

- ✓ Attendu : aucune prestation des seeds ne dépasse 1000 CHF en FORFAIT_INTEGRATION (P-005 = 600) → pas de nouveau warning. Pour le déclencher : passer le plafond à `500` → P-005 devient suspecte. **Aucun fichier SQL modifié.**

4.  Retirer la ligne du contrat, rebuild.

Corrigé PII (§5.4) :

```
cd dbt
dbt build --select stg_individu -q
docker exec -it pocs-postgres psql -U user -d pocs \
  -c "select nom, prenom, navs from silver.stg_individu limit 3;" # → MASQUÉ
partout
dbt build --select stg_individu -q --vars '{current_role: dpo}'
docker exec -it pocs-postgres psql -U user -d pocs \
  -c "select nom, prenom, navs from silver.stg_individu limit 3;" # → en clair
dbt build --select stg_individu -q # ↺ re-masquer
!
```

cd ..

⚠ Ne pas oublier le dernier build : la vue reste telle que le dernier build l'a définie. Chaque bascule est tracée dans `audit.pii_access_log` (voir S10).

SCD2 (§5.6) :

```
make snapshot
docker exec -it pocs-postgres psql -U user -d pocs \
  -c "select id, permis, dbt_valid_from, dbt_valid_to from
snapshots.snap_permis_individu order by id;"
# Provoquer un changement de statut puis re-snapshoter :
docker exec -it pocs-postgres psql -U user -d pocs \
  -c "update bronze.permis set permis='B', date_debut='2026-07-01' where id=5;" #
Ivan Petrov F→B
make snapshot
docker exec -it pocs-postgres psql -U user -d pocs \
  -c "select id, permis, dbt_valid_from, dbt_valid_to from
snapshots.snap_permis_individu where id=5 order by dbt_valid_from;"
```

✅ Attendu : 2 versions pour id=5 — l'ancienne (permis F) close, la nouvelle (permis B) ouverte. ↺ `make seed && make snapshot` pour restaurer.

S6 — Gold

```
cd dbt && dbt build --select marts && cd ..
```

Corrigé de l'exercice « lire comme un auditeur » (§6.4) :

```
docker exec -it pocs-postgres psql -U user -d pocs -c "
select numero_individu, type_prestation, date_prestation,
       is_conforme_couverture as couv, is_alerte_senior as senior,
       is_alerte_lamal_age as lamal, is_verse_apres_deces as deces,
       is_migrant_sans_permis as migrant
from gold.fct_prestations
where not is_conforme_couverture or is_alerte_senior or is_alerte_lamal_age
       or is_verse_apres_deces or is_migrant_sans_permis
order by numero_individu;"
```

✓ Attendu : 6 lignes — | Ligne | Explication métier | |---|---| | IND-999 / FORFAIT_ENTRETIEN | inconnu du référentiel → pas de couverture possible (RG_23 + RG_18) | | IND-010 / FORFAIT_ENTRETIEN juin | son permis N est expiré et le Z est invalide → non couverte (RG_18) | | IND-007 / FORFAIT_ENTRETIEN | 72 ans : relève de l'AVS/AI, pas du forfait (RG_20) | | IND-004 / PRIME_LAMAL | Emma a 13 ans : hors tranche 18-25 (RG_25) | | IND-012 / FORFAIT_ENTRETIEN | versé le 05/03, décédé le 15/02 (RG_26) | | IND-001 / AIDE_URGENCE | prestation « migrant » sans aucun permis (RG_27) |

Démo contrat de schéma (§6.3) :

```
# Renommer montant_par_jour → cout_journalier dans
dbt/models/marts/fct_prestations.sql (2 endroits)
cd dbt && dbt build --select fct_prestations
```

✓ Attendu : échec immédiat [This model has an enforced contract... definition_error: montant_par_jour] avec le diff des colonnes.  Annuler la modification, rebuild.

S7 — Tests et WAP

```
# 7.1 – La pyramide, étage par étage
make unit
```

✓ Attendu : `PASS=3` (rg22_cout_journalier..., rg18_couverture..., rg20_rg26...), en ~1 s, **avant tout build**.

Corrigé de l'exercice « casser la macro » (§7.2) :

```
# Dans dbt/macros/normalize_open_period.sql : remplacer var("date_fin_ouverte") par
'2020-01-01'
make unit
```

✓ Attendu : `rg18_couverture_active_et_inactive` **FAIL** — IND-T2 devient non conforme (sa couverture « ouverte » se ferme en 2020). 🔄 Restaurer, `make unit` → `PASS=3`.

```
make build          # mode alerte : PASS=71 WARN=18 ERROR=0
make test-strict    # fail-fast
```

✓ Attendu pour `test-strict` : **échec** (exit ≠ 0) — les RG bloquantes (06, 09, 10, 13, 14, 16, 17, 23, 26, 28, 29) passent de WARN à ERROR sur les seeds piégés. *C'est le comportement voulu* : en production, les seeds n'ont pas d'anomalies, donc strict = vert.

```
make audit
```

✓ Attendu : le tableau des **15 RG** avec anomalies, chacune à 1 sauf RG_18 à 2 (cf. cahier §7.4).

```
make report          # rapport Elementary
```

✓ Attendu : `edr` génère `elementary_report.html` (l'ouvrir : résultats par modèle, historique des runs, anomalies RG_31/32).

WAP Nessie (§7.5) :

```
curl -s -X POST http://localhost:19120/api/v1/trees/branch \
  -H "Content-Type: application/json" \
  -d '{"name": "feature/demo-wap", "sourceRefName": "main"}'
curl -s http://localhost:19120/api/v1/trees | python3 -m json.tool # la branche
existe

curl -s -X DELETE "http://localhost:19120/api/v1/trees/branch/feature%2Fdemo-wap?
expectedHash=$(curl -s http://localhost:19120/api/v1/trees/tree/feature%2Fdemo-wap | python3 -
c 'import sys,json;print(json.load(sys.stdin)["reference"]["hash"])')"
```

✓ Attendu : création instantanée (Zero-Copy), la branche apparaît dans la liste, suppression propre. Le build dbt *sur* la branche nécessite la cible lakehouse (dbt-dremio/Trino) — hors périmètre atelier, expliqué au tableau.

JOUR 3

S8 — CI/CD

pre-commit (local, sans GitHub) :

```
git init 2>/dev/null; git add -A # si le repo n'est pas encore
initialisé
pre-commit run --all-files
```

✓ Attendu : check-yaml, end-of-file-fixer, trailing-whitespace, sqlfluff et dbt parse passent (sqlfluff peut remonter du style : c'est l'occasion de montrer --fix).

Les 3 exercices PR (§8.5) — nécessitent le repo poussé sur GitHub :

```
git remote add origin <votre-repo> && git push -u origin main
# Activer : Settings → Pages → Source = GitHub Actions
```

1. **PR verte** : brancher, ajouter - AIDE_TRANSPORT au domaine dans dbt_project.yml, pousser, ouvrir la PR. ✓ CI verte ; dans l'artefact dbt-docs, agg_prestations_mensuelles a une colonne mnt_AIDE_TRANSPORT.
2. **PR rouge** : brancher, retirer - LOYER du domaine. ✓ Le job audit échoue : in_contract_domain sur type_prestation trouve les LOYER des seeds ; et le « test des tests » resterait rouge de toute façon.
3. **PR sournoise** : brancher, supprimer le test dbt_expectations.expect_column_values_to_be_between (RG_14) de staging/schema.yml. ✓ lint/build/tests **verts**, mais l'étape « Contrôle fail-fast » échoue : dbt build --vars '{rg_severity: error}' passe (plus rien ne bloque le montant -50) → le garde-fou exit 1.

Sans GitHub (démonstration locale équivalente de la PR sournoise) :

```
# après avoir retiré le test RG_14 du schema.yml :  
cd dbt && if dbt build --vars '{rg_severity: error}' >/dev/null 2>&1; \  
  then echo "GARDE-FOU: les RG bloquantes sont inopérantes (CI serait ROUGE)"; \  
  else echo "ok"; fi; cd ..  
# 🔄 restaurer le test
```

⚠ Nuance : avec le seul test RG_14 retiré, d'autres RG bloquantes échouent encore en strict, donc le garde-fou local dit « ok ». Pour la démo pure, lancer le strict **ciblé** : `dbt build --select stg_prestation --vars '{rg_severity: error}'`.

S9 — Airflow

```
docker compose --profile orchestration up -d
docker logs benefits-airflow 2>&1 | grep "Password for user"
```

✓ Attendu : login `admin` + mot de passe affiché. Ouvrir `http://localhost:8090` → DAG `benefits_lakehouse` (pause OFF pour l'activer).

1. **Déclencher** (► Trigger DAG) → vue Graph : `dbt_deps` → `source_freshness` → `dbt_seed` → `dbt_build` → `dbt_snapshot` → `dbt_test_singular` → `dbt_docs_generate`, tout vert (freshness peut être orange/failed si les seeds datent → `make freshen` d'abord, ou accepter l'échec comme démo RG_02 !).
2. **Panne/reprise** : `docker stop pocs-postgres` → Trigger → `dbt_seed` échoue après 1 retry (5 min — pour la démo, réduire `retry_delay` dans le DAG à `timedelta(seconds=30)`) → `docker start pocs-postgres` → clic sur la tâche rouge → **Clear** → elle repart et la suite s'enchaîne sans rejouer `dbt_deps` / `source_freshness`.

S10 — Lignage, doc, DPO

```
# Lignage
export OPENLINEAGE_URL=http://localhost:5000 OPENLINEAGE_NAMESPACE=benefits
make lineage
```

✓ Attendu : `dbt-ol` wrappe le build et pousse les événements ; sur `http://localhost:3000`, namespace `benefits`, chercher `fct_prestations` → onglet graphe : bronze → staging → marts. (Si l'UI reste vide : la variable n'était pas exportée dans le shell de `make`.)

```
# Documentation : les trois surfaces
make docs                                # 1. site : http://localhost:8087
docker exec -it pocs-postgres psql -U user -d pocs -c "\d+ gold.fct_prestations" #
2. commentaires SQL
   cd dbt && dbt ls --select +exposure:dashboard_pilotage_prestations && cd ..    #
3. analyse d'impact
```

✓ Attendu (3) : la liste seeds → staging → marts dont dépend le dashboard — la réponse outillée à « peut-on supprimer ce modèle ? ».

```
# RG_33 – minimisation (démonstration négative)
# Ajouter « , 'x' as email » dans le select final de dim_calendrier.sql, puis :
cd dbt && dbt build --select dim_calendrier && dbt test --select
rg33_minimisation_pii_gold
```

✓ Attendu : le test **échoue** (colonne `email` détectée dans le catalogue gold). 🔄 retirer, rebuild, retester → PASS.

```
# RG_34 – journal des accès
docker exec -it pocs-postgres psql -U user -d pocs \
-c "select executed_at, commande, role_courant, pii_exposees, nb_noeuds
    from audit.pii_access_log order by executed_at desc limit 5;"
```

✓ Attendu : une ligne par commande dbt de la journée ; `pii_exposees=t` uniquement pour les builds lancés avec `current_role: dpo` (S5).

```
# RG_35 – droit à l'oubli, cycle complet
make forget IND=IND-011
docker exec -it pocs-postgres psql -U user -d pocs \
  -c "select count(*) from silver.stg_individu where numero_individu='IND-011';" \
  -c "select * from audit.rgpd_forget_log;"
make seed && make build      # 🔄 restaurer les données d'atelier
```

✅ Attendu : purge des 4 tables bronze journalisée, 0 ligne en silver après propagation, la demande tracée **sans PII** dans `rgpd_forget_log`. Effet de bord pédagogique : IND-011 portait les anomalies RG_06/RG_28 → après l'oubli, `make audit` en affiche 13 au lieu de 15.

S11 — Restitution

- **Metabase** (<http://localhost:3002>) : Admin → Databases → Add → PostgreSQL
(`host.docker.internal`, port `5433`, db `pocs`, user/password) → construire les 5 cartes du cahier §11.2 sur le schéma `gold`. La requête du tableau d'alertes DPO est celle du corrigé S6 ci-dessus.
- **Dremio** (<http://localhost:9047>) : Add Source → Nessie (`http://nessie:19120/api/v2`, auth None)
+ S3 (MinIO `http://minio:9000`, clés du `.env`) — utile seulement si vous avez alimenté Iceberg via NiFi/consumer en S4.

S12 — Mini-projet : corrigé complet du sujet A (« Rétention »)

Le chemin exact attendu d'un binôme (à adapter pour B/C/D) :

1. Contrat métier — `dbt/contracts/prestation.yml` :

```
- id: RG_36
  nom: Rétention des prestations
  intention: >
    Au-delà de la durée légale d'archivage, les prestations ne doivent
    plus apparaître dans les zones d'analyse (Gold).
  gravite: alerte # flag, pas filtre – choix à justifier (voir 5.)
```

2. Contrat de données — `dbt/dbt_project.yml`, sous `vars.contracts.prestation` :

```
retention_annees: 10
```

3. Implémentation — dans `fct_prestations.sql` (CTE `enrichi`, avec les autres flags) :

```
-- RG_36 : prestation au-delà de la durée légale de rétention
(p.date_prestation
 < current_date - make_interval(years =>
    {{ get_contract('prestation', 'retention_annees') }})
) as is_hors_retention,
```

... l'ajouter au `select` final **et** au contrat de schéma `marts/schema.yml` :

```
- name: is_hors_retention
  data_type: boolean
  description: RG_36 – prestation plus ancienne que la durée de rétention du
  contrat.
  tests:
    - accepted_values:
      values: [false]
      config: { severity: warn, tags: ["rg36"] }
```

4. Seed piégé + cas légitime voisin — `dbt/seeds/prestation.csv` :

```
P-022,IND-001,AIDE_MATERIELLE,300.00,2014-01-01,2014-01-31,2014-01-05,2026-07-01
06:00:00,postgres_src,batch-001
P-023,IND-001,AIDE_MATERIELLE,300.00,2020-01-01,2020-01-31,2020-01-05,2026-07-01
06:00:00,postgres_src,batch-001
```

(P-022 : 2014 → hors rétention ; P-023 : 2020 → dans la fenêtre, ne doit PAS déclencher.)

5. Vérifier :

```
cd dbt && dbt build --select fct_prestations
docker exec -it pocs-postgres psql -U user -d pocs \
  -c "select pk_prestation is not null, date_prestation, is_hors_retention
      from gold.fct_prestations where date_prestation < '2021-01-01';"
```

✅ Attendu : P-022 flaguée `true`, P-023 `false`, 1 warning de plus au build. Point de débat noté dans la grille : *flag vs filtre* — un `where` ferait chuter la réconciliation RG_24 et « perdrait » des lignes silencieusement ; le flag garde l'anomalie visible et laisse la purge à un processus explicite (comme RG_35).

6. Audit — ajouter dans `analyses/audit_tableau_de_bord_rg.sql` :

```
union all
select 'RG_36 – Prestation hors durée de rétention', count(*)
from {{ ref('fct_prestations') }} where is_hors_retention
```

7. PR + CI : brancher, commit (pre-commit doit passer), pousser, PR → CI verte, « test des tests » compris : en strict, RG_36 est `warn` donc ne bloque pas — question bonus de la restitution : *faudrait-il qu'elle bloque ?*

🔄 **Reset post-mini-projet** : `git checkout main && make seed && make build`.

Récapitulatif des commandes de reset

Situation	Commande
Données modifiées/purgées	<code>make seed && make build</code>
Snapshots pollués par les démos	<code>docker exec -it pocs-postgres psql -U user -d pocs -c "drop schema snapshots cascade;" && make snapshot</code>
Journaux d'audit à vider (nouveau groupe)	<code>docker exec -it pocs-postgres psql -U user -d pocs -c "drop schema audit cascade;"</code> (recréé au prochain run)
Fraîcheur en erreur	<code>make freshen</code>
Tout recommencer	<code>docker compose --profile "*" down -v && docker compose --profile "*" up -d</code> puis \$0